From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Reza Rahimi
JazzX AI
Los Altos, CA, USA
reza.rahimi@jazzx.ai

Abstract

Tool-using agents often repeat the same read-only evidence-gathering prefix while paying for a model decision at every tool boundary. Replacing such prefixes is not ordinary caching: tool arguments may depend on prior observations, nominal reads may hide side effects, and a replay-correct program may still change the final answer. We present guarded agentic compaction (GAC), a trace-to-program compiler that reconstructs typed provenance, enforces effect and position barriers, synthesizes from a bounded DSL, and admits compiled prefixes only under a compile-or-retire selective-risk test. On three public-record GitHub workflow families with live provider calls, compiled programs achieve 90/90 exact held-out contracts versus 89/90 for unchanged agents while reducing provider requests by 50.0–75.0%, total tokens by 39.5–81.4%, observed latency by 51.7–73.0%, and estimated cost by 32.0–75.3%. The same protocol transfers across repositories on a narrower time-forward pull-request task: a conservative five-repository extension yields 120/120 exact held-out pairs on four repositories and retires the fifth, and a balanced three-repository rerun reaches 180/180 exact pairs with 66.7% fewer provider requests. On NESTFUL and API-Bank every recurrent family retires even though provenance, synthesis, and replay all succeed, showing that recurrence and replayability are insufficient without admission support. Hand-written programs remain competitive when the workflow is already known, so the claim is automatic discovery and conservative lifecycle management, not runtime dominance over manual code.

1 Introduction

The standard agent loop pays for a model decision at every tool boundary: model, tool, model, tool, and so on (Yao et al., 2023). That flexibility is valuable while a workflow is novel. At scale, however, mature agents often revisit the same read-only evidence path with the same data dependencies. In those settings, repeating the model decision adds latency, tokens, and cost without necessarily adding useful adaptation.

Removing that decision is not ordinary caching. A tool argument may depend on a specific earlier observation, a repeated sequence may cross an approval or write, an apparently read-only tool may hide an undeclared effect, and a program that replays the same tool outputs may still change the downstream answer. The problem is therefore not just "find recurring traces." The problem is to decide when a recurrent model-mediated region can be replaced by a deterministic program and when it should be refused.

Consider the smallest version of this decision in the benchmark. A NESTFUL record asks for the percentage increase from 50 dollars to 60 dollars and records `subtract(60,50) -> divide(result,50) -> multiply(result,100)`. The trace is executable and every later argument is grounded in an earlier value, so a recurrence-only optimizer would have a plausible macro. The benchmark even supplies successful replay: across 32 attempted families, 24 held-out windows pass, 12 abstain, and none are wrong. But after provenance and group filtering the strongest candidate family has only 26 independent groups, while the configured exact gate needs 92 zero-violation groups ($\alpha = .05$, $\delta = .10$, eleven thresholds). GAC therefore retires every family and keeps the

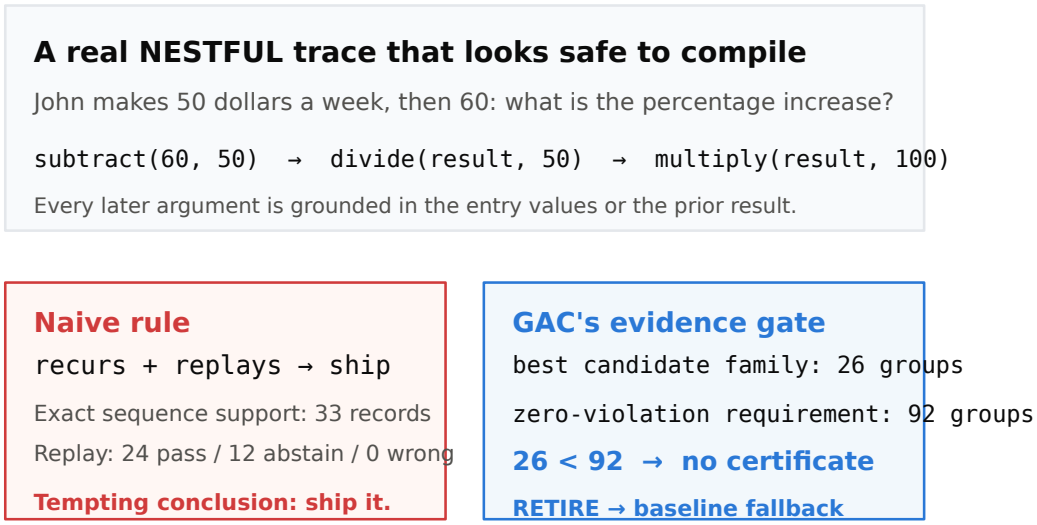

Figure 1: The recurrence-versus-admission distinction on NESTFUL. This is provider-free structural compiler evidence, not a live planning or production-safety result.

unchanged agent. The example makes the distinction concrete: recurrence says what may be compilable; evidence determines whether replacement is justified.

The benchmark also shows why tool replay is not full substitution evidence. In an earlier 18-record GitHub study, a more aggressive three-read artifact had 45/45 clean tool replays but only 17/18 exact downstream answers: one Markdown URL was lost from the continuation. A checked continuation renderer recovered 18/18 in provider-free replay, but the paper treats that as a detection mechanism rather than a live safety claim.

Framed this way, the closest precedent is not prompt optimization but the tracing just-in-time compiler. Dynamo (Bala et al., 2000), trace-based type specialization (Gal et al., 2009), and meta-tracing (Bolz et al., 2009) all detect a hot path, compile it behind a *guard*, and deoptimize to the interpreter when the guard fails (Hölzle et al., 1992). Agent traces admit the same structure — hot-trace detection becomes family mining, guard synthesis becomes an entry contract, and deoptimization becomes fallback to the unchanged agent — but they add three obligations a JIT does not face. Tool arguments must be shown to be *grounded* in observable state rather than invented by the model; declared *effects* must permit speculation, because an agent's "interpreter" can call the outside world; and the residual error rate of a compiled region is statistical, so it needs a finite-sample bound rather than a soundness proof.

We study these obligations through *guarded agentic compaction* (GAC), a compile-or-retire trace-to-program system for recurrent read-only prefixes. The method normalizes executions into a typed episode representation, reconstructs value provenance for tool arguments, mines recurrent prefix families, synthesizes bounded programs, induces runtime guards and verifiers, and admits an artifact only when a finite-sample selective-risk calculation supports it. Otherwise the baseline agent is retained unchanged (fig. 2). The core contribution is thus not "agent compilation" in general, but an admissibility argument for trace-derived specialization.

The evidence is deliberately multi-sided. On three distinct GitHub workflow families with live provider calls, compiled programs preserve or improve exact held-out task outcomes while reducing requests, tokens, latency, and cost; a narrower time-forward task then tests transfer across repositories and retains both success and compile-time retirement. NESTFUL and API-Bank show the opposite regime, in which recurrent executable traces still fail to justify deployment. We also retain the strongest practical counterpoint: when a workflow is already known, hand-written programs can be competitive or tied, so any claim of automatic superiority would be incorrect.

The paper makes three contributions:

1. A guarded specialization pipeline (Alg. 1) that combines typed provenance, effect and position barriers, bounded synthesis, runtime verification, and finite-sample compile-or-retire admission (Alg. 2, proposition 1).
2. A real-provider evaluation across three public-record workflow families, plus a narrower cross-repository, time-forward extension that keeps both successful artifacts and principled retirements.
3. A negative-result boundary showing that recurrence and replayability do not themselves license deployment: NESTFUL and API-Bank retire under the same admission logic, and manual programs constrain any runtime-dominance claim.

2 Problem Formulation

Episodes and candidate regions. An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t (model requests and responses, tool calls and results, handoffs, guardrails, approvals, errors, commit boundaries), and an observable outcome y. Each tool f carries an application-owned effect declaration $\epsilon(f)$ and capability flags such as *speculatable*; an unknown effect is not a read.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is an expression over entry state or prior results inside R,

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j):\quad u = g_{j,u}\big(z, o_{<j}\big), \qquad g_{j,u} \in \mathcal{L}, \tag{1}$$

for a closed, bounded transform library $\mathcal{L}$; and *effect-admissible* when every tool in R is declared read-only, speculatable, replayable, and inside one principal and isolation partition. Handoffs, approvals, writes, errors, and unknown effects are hard barriers, not costs to be traded away. Because the deployed runtime resolves artifacts at the initial model boundary, the compiler also imposes a *position invariant*:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

This is not aesthetic. A suffix program dispatched at entry can reorder or duplicate calls the agent has already made — a semantic mismatch between compilation-time and resolution-time position rather than a tuning failure, and a pilot that violated eq. (2) produced exactly that fault (appendix E).

Selective optimization objective. The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: a deterministic program P, hard guard H, output verifier V, nonconformity score q, threshold η, and manifest pins M. On a future episode with entry state z, dispatch is *selective*,

$$d_A(z) = \mathbf{1}\{\, H(z) = 1 \ \wedge\ q(z) \le \eta \ \wedge\ M \text{ matches} \,\}, \tag{3}$$

and any failed precondition runs the unchanged baseline agent B. If execution fails before a commit and staging proves the attempt clean, B runs; a dirty failure after a commit is an incident, not a fallback. Writing $L(A, z) \in \{0, 1\}$ for a wrong compiled execution *after* dispatch and $C(\cdot, z)$ for execution cost, we seek

$$\max_{A}\ \mathbb{E}\big[d_A(z)\,\{C(B, z) - C(A, z)\}\big] \quad \text{s.t.} \quad \Pr[L(A, z) = 1 \mid d_A(z) = 1] \le \alpha. \tag{4}$$

Two properties shape everything that follows. The constraint conditions on dispatch, so abstention is free and only post-dispatch errors are charged — the classical reject option (Chow, 1970) applied to a compiler rather than a classifier — and returning *no* artifact is feasible, so RETIRE is a legitimate output rather than a failure mode. Equation (4) is a design target: the implementation returns the first family that survives every stage of Alg. 1, and we prove no bound on the gap to the best one.

3 Guarded Agentic Compaction

Figure 2 and Alg. 1 give the same six stages at two levels of detail. We describe each and state what it can refuse.

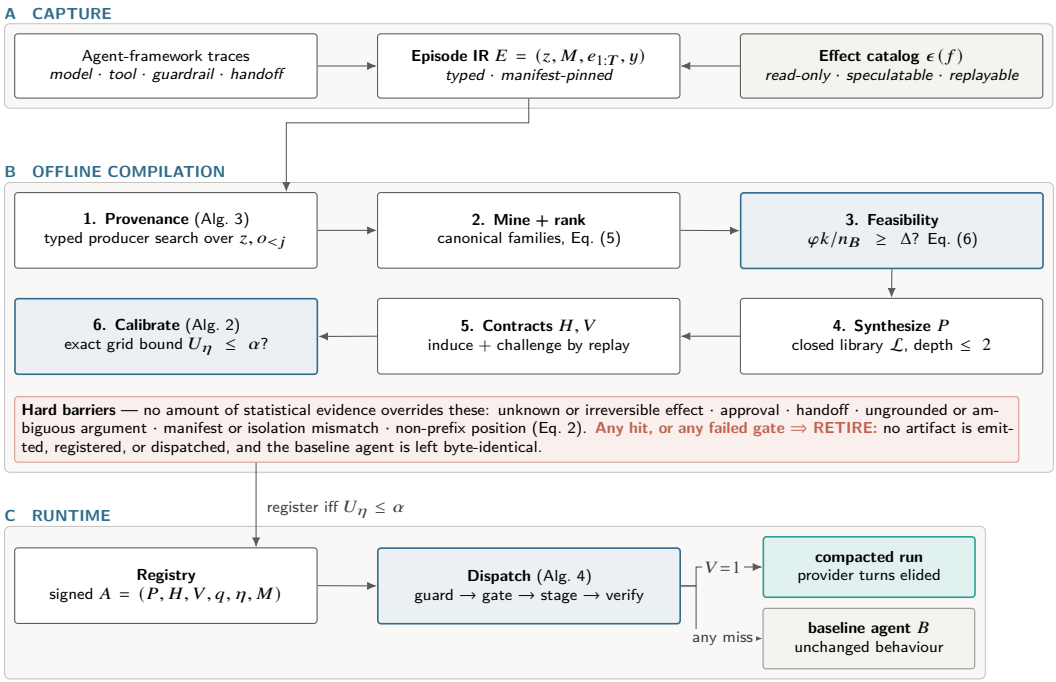

Figure 2: Architecture of GAC. **(A)** Capture normalizes traces into one typed Episode IR against a declared effect catalog. **(B)** Offline compilation is a cascade of independent rejection points; the shaded band lists barriers no statistical evidence can override. **(C)** At runtime exactly one terminal edge compacts; the rest return the unchanged agent.

1. Typed provenance. A capture adapter normalizes framework traces into the Episode IR; manifests pin prompt, policy, guardrail, tool, model, SDK, tracer, entry-contract, and effect-catalog identities, and episodes with different compatibility keys are partitioned before any learning. For each call-argument slot, the program-argument trace graph (PATG, Alg. 3) searches entry-state and prior-result paths for typed values reachable by a bounded transform — execution provenance in the database sense (Cheney et al., 2009), specialized to tool arguments. Two outcomes block a region rather than guessing: a slot with *no* witness is a genuine model decision, and a slot with more than κ witnesses is ambiguous.

2. Region mining and ranking. The miner enumerates model-boundary-to-result windows of at most $w_{\max}$ tool calls, qualifies effects and live-ins, and hashes tool signatures, dependency topology, run shape, and live-in/live-out shape while abstracting literal values. Families require support across independent scenario *groups* rather than raw event frequency, which prevents a single chatty scenario from manufacturing recurrence. Survivors are ranked by a minimum-description-length trade-off that pays for expected savings and charges for heterogeneity, effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\, \bar{k}_F\, c_m}_{\text{expected saving}} - \lambda_1 H(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{5}$$

with s_F the group support, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $H(F)$ the Shannon entropy over canonical variants inside the family; without λ_1 the ranking prefers a heterogeneous family whose single canonical form fits none of its members. Equation (5) affects only *which* family is tried first — no admission decision depends on it.

3. Feasibility ceiling. Before any synthesis runs, an estimator bounds what the compiler could achieve with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B}, \qquad \text{necessary:} \quad \varphi\, \rho\, k \;\geq\; \Delta\, n_B, \tag{6}$$

Algorithm 1 COMPILE — the compile-or-retire cascade. Every exit is either a registered artifact or an attributed RETIRE, so rejections stay in the denominator. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Input: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; budgets α, δ; grid Λ; feasibility target Δ
Output: artifact $A = (P, H, V, q, \eta, M)$, or RETIRE with an attributed reason

$\mathcal{E} \leftarrow \textsc{Qualify}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \textsc{PartitionBy}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \textsc{GroupedSplit}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $G \leftarrow \textsc{BuildPatg}(\mathcal{T}, \Theta)$ ▷ Alg. 3: block ungrounded or ambiguous slots
  $\mathcal{F} \leftarrow \textsc{Canonicalize}(\textsc{MineRegions}(G, C, w_{\max}))$, keeping only cross-group support
  if $\varphi k / n_B < \Delta$ **then** (CALLER)
    return RETIRE **(infeasible ceiling, Eq. 6)** ▷ run before paying for synthesis
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (5) **do**
    $P \leftarrow \textsc{Synthesize}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then continue** ▷ ungroundable slot
    $(H, V) \leftarrow \textsc{InduceContract}(F)$; **if** $\neg\textsc{ReplayAndChallenge}(P, V, \mathcal{D})$ **then continue** ▷ counterexample
    $q \leftarrow \textsc{FitScore}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \textsc{Calibrate}(q, H, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then continue** ▷ Alg. 2
    return $\textsc{Package}(P, H, V, q, \eta, M)$ with evidence and lifecycle
  end for
end for
return RETIRE

with n_B baseline model requests per episode, φ the fraction of episodes holding an eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Since eq. (6) assumes $\rho = 1$, no measured reduction can exceed it, and it is met with equality exactly when nothing abstains and nothing fails. Reporting it first makes a decisive negative answer cheap: a workload whose ceiling sits under the target can be declined without building a compiler for it.

4–5. Bounded synthesis and contracts. Synthesis searches a 23-operator library at depth at most two, covering path selection, indexing, string normalization, arithmetic, comparisons, bounded collection operations, and narrow loops; bindings are ranked by stability across groups, and a group-wise refit rejects bindings that exploit row-level coincidence. The hypothesis space is deliberately small enough to read: the approach resembles programming-by-example (Gulwani, 2011) and bounded sketching (Solar-Lezama et al., 2006), and it does *not* generate arbitrary Python. Hard guards then constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins; verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are likely invariants (Ernst et al., 2001), not proofs for all future inputs — which is why a statistical gate follows them rather than replacing them.

6. Exact risk-gated admission. Algorithm 2 decides deployment. The score q is fitted on *dev* groups only — never on calibration groups — from entry-observable features such as missing fields, hull margin, temporal drift, and provenance ambiguity, then frozen together with a finite threshold grid Λ. For each $\eta \in \Lambda$ the compiler counts admitted calibration groups (n_η) and those containing at least one post-dispatch violation (k_η), and inverts the one-sided exact binomial test (Clopper & Pearson, 1934) at a Bonferroni-corrected level:

$$U_\eta = \text{Beta}^{-1}\Big(1 - \tfrac{\delta}{|\Lambda|};\ k_\eta + 1,\ n_\eta - k_\eta\Big), \qquad U_\eta\Big|_{k_\eta=0} = 1 - \Big(\tfrac{\delta}{|\Lambda|}\Big)^{1/n_\eta}. \tag{7}$$

It admits the largest-coverage threshold with $U_\eta \le \alpha$ and retires the family otherwise — a fixed-grid instance of the learn-then-test discipline (Angelopoulos et al., 2022; Bates et al., 2021). The closed form on the right is the operative constraint in our experiments: at $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$, a zero-violation family still needs $n_\eta \ge 92$ admitted groups. The default output is refusal, not emission.

Proposition 1 (Per-candidate selective-risk admission)**.** *Fix one candidate program, hard guard, verifier, score, and finite threshold grid Λ before calibration. If admitted calibration groups are i.i.d.,*

Algorithm 2 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Input: frozen score q; hard guard H; calibration groups $\mathcal{K}$; grid Λ; budgets α, δ
Output: threshold η with a simultaneous guarantee, or $\perp$ (RETIRE)

$\mathcal{A} \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs
for all $\eta \in \Lambda$ **do**
  $K_\eta \leftarrow \{\, g \in \mathcal{K} : \exists z \in g,\ q(z) \le \eta \ \wedge\ H(z) = 1 \,\}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  $k_\eta \leftarrow |\{\, g \in K_\eta : g \text{ contains a violation} \,\}|$ ▷ *wrong after dispatch* only, never abstention
  $U_\eta \leftarrow \mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|};\ k_\eta + 1,\ n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\mathcal{A} \leftarrow \mathcal{A} \cup \{(n_\eta/|\mathcal{K}|, \eta)\}$ ▷ Eq. (7)
end for
return $\perp$ if $\mathcal{A} = \emptyset$, else $\arg\max_{(\mathrm{cov},\eta)\in\mathcal{A}}\ \mathrm{cov}$ ▷ largest coverage among certified thresholds

then with probability at least $1 - \delta$ *every* $\eta \in \Lambda$ *satisfies* $r_\eta \le U_\eta$*, where* r_η *is the selective risk at* η *and* U_η *is given by eq.* (7)*. Hence the data-selected threshold satisfies the target selective risk of eq.* (4) *whenever its upper bound is at most* α*.*

Appendix A gives the proof. The guarantee is deliberately narrow: it is *per fixed candidate*, not compiler-wide over the adaptive candidate search Alg. 1 actually performs, and it conditions on group exchangeability rather than establishing it (section 7). The scientific point is that guarded deployment requires an explicit evidence boundary, not just a recurrent trace and a successful replay.

4 EXPERIMENTAL SETUP

Because the claim is an *admissibility* claim, no single benchmark can support it. We use three evidence classes: does guarded specialization preserve outcomes while saving work on real records (§5.1); does it transfer beyond one repository under time-forward evaluation (§5.2); and does the admission rule actually refuse when evidence is thin (§5.3)?

Primary live workflow transfer. Three public-record GitHub workflow families over one revision-pinned snapshot: issue-type routing, pull-request outcome audit, and backlog-attention routing. Each has a distinct tool vocabulary and an exact three-class contract, and uses 132 discovery and 30 balanced held-out records, with live provider calls.

Cross-repository time-forward extension. A narrower pull-request outcome task over five frozen public repositories, with two read-only tools and a strict time-forward preflight: held-out records are strictly newer than discovery records, and candidate selection is sealed before any provider call. The original cohort uses 116 discovery and 30 held-out records per repository, freezes one candidate before calibration, and retains repositories with no admitted artifact. Because it is class-skewed on some mirrors, we also run a balanced rerun on three repositories with 120 discovery and 60 held-out pull requests each, evenly split across `open`, `merged`, and `closed_unmerged`.

External refusal benchmarks. NESTFUL and API-Bank test what happens once a valid, executable trace exists but evidence is still insufficient. NESTFUL provides 1,415 executable post-trace episodes with nested producer references (Basu et al., 2025); API-Bank contributes 389 recorded tool calls over 49 tools (Li et al., 2023). Both retain argument-level producer structure that trajectory-only suites discard; they measure provenance reachability, synthesis, replay, and admission, not live planning quality.

Conditions, metrics, and statistics. Each GitHub family runs three conditions on identical held-out records: the *unchanged baseline* B, the *compiled* artifact under eq. (3), and a *hand-written* pre-model program authored with full knowledge of the workflow — the comparator most likely to falsify a runtime-dominance claim, so it appears in the main results rather than the appendix. The primary quality endpoint is exact task-contract satisfaction; resource endpoints are provider requests, provider-visible tool interfaces, total tokens, observed wall latency, and estimated cost. All comparisons are paired on the same records: binary quality uses exact McNemar tests (McNemar, 1947) and

Table 1: Primary live-provider results ($n = 30$ held-out records per family). "Interfaces" counts provider-visible tool interfaces. All conditions see identical records, model, and grader; *Manual* is hand-written with full knowledge of the workflow.

	Exact held-out contract			Reduction of compiled vs. baseline (%)				
Family	Base	Compiled	Manual	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30	**30/30**	30/30	50.0	0.0	39.5	51.7	32.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	66.7	80.8	73.0	75.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	66.3	81.4	68.9	75.1
Weighted total ($n = 90$)	89/90	**90/90**	90/90	66.6	44.2	63.1	64.2	58.7

one-sided Clopper–Pearson bounds (Clopper & Pearson, 1934), and paired resource differences use 10,000-sample bootstrap intervals (Efron & Tibshirani, 1993) with two-sided Wilcoxon signed-rank tests (Wilcoxon, 1945). Rows whose paired difference is deterministic by construction are reported without a p-value. Secondary p-values are descriptive and unadjusted; the only multiplicity correction carrying a guarantee is the Bonferroni term inside eq. (7). All admission runs use $\alpha = 0.05$, $\delta = 0.1$, and $|\Lambda| = 11$; appendix C lists the rest.

5 Results

5.1 Primary GitHub workflow families

The result transfers across all three families. Compiled programs reach 90/90 exact held-out outcomes versus 89/90 for unchanged agents (table 1). In absolute terms the 90 records cost the baseline 359 requests, 294,677 tokens, 537.5 s, and \$0.0684 against 120, 108,839, 192.4 s, and \$0.0283 compiled — reductions of 66.6%, 63.1%, 64.2%, and 58.7%. The per-family spread matters more than the aggregate: the issue-type task supports only a conservative two-read prefix, while the two deeper families approach 75% request reduction. For the deepest family the measured 75.0% reduction *equals* the feasibility ceiling $\varphi k/n_B$ at $\varphi = 1$, $k = 3$, $n_B = 4$ — the signature of a task that fully determines its own region, which is a property of the workload rather than evidence about the compiler.

These differences are large relative to their sampling error: the PR-outcome family saves 2196.7 tokens per record (95% bootstrap CI $[2182.5, 2211.1]$, Wilcoxon $p = 1.7 \times 10^{-6}$) and the backlog family 2315.0 (CI $[2224.1, 2366.0]$); appendix D gives every endpoint. Request counts are excluded from significance testing because every pair removes exactly the same number of turns by construction.

Quality preservation is strong but should be stated precisely. Across the 90 paired held-out records there is no compiled-only failure, so the one-sided 95% exact upper bound on compiled-only discordance is 3.3%; the single baseline-only failure gives an exact McNemar $p = 1.0$. The paired evidence is consistent with no quality difference in either direction, which is a bound on the observed sample rather than an equivalence claim, and it does not override the per-candidate scope of proposition 1.

Each admitted artifact carries an explicit certificate: for all three families the gate admitted $n_\eta = 92$ calibration groups with $k_\eta = 0$ violations, giving $U_\eta = 0.0498 \le \alpha = 0.05$ at coverage 1.0 — exactly the minimum eq. (7) certifies at $\delta = 0.1$. Every family sits *at* the admission boundary, which is what makes section 5.3 inevitable.

The strongest practical comparator narrows the contribution further. Hand-written programs also reach 90/90 exact outcomes, and on the issue-type family the manual program is cheaper than the compiled one (\$0.0164 vs. \$0.0178). The live evidence therefore supports automatic trace-grounded discovery, guarded admission, and lifecycle management — not a claim that compiled programs universally beat correct manual implementations.

Table 2: Cross-repository time-forward results on the narrower two-read pull-request outcome task. Held-out records are strictly newer than discovery records and selection is sealed before any provider call. The 5-repo cohort retires `pytorch/pytorch`; a fixed template is at least as efficient in both settings.

		Exact task contract			Reduction vs. baseline (%)			
Setting	Repos	Discovery	Held-out	Classes	Req.	Tokens	Latency	Cost
5-repo core	4/5	580/580	120/120	skewed	44.4	52.4	49.4	48.6
3-repo rerun (balanced)	3/3	360/360	180/180	balanced	66.7	78.4	60.7	72.7

Table 3: Where refusal happens. Both benchmarks clear provenance, synthesis, and replay, then retire at calibration with *zero* wrong held-out executions: the binding constraint is sample size, not correctness. Stages refer to Alg. 1.

Stage of Alg. 1	NESTFUL	API-Bank
Episodes / recorded traces	1,415	212
Tools compilable / effect-blocked	—	21 / 28
Dependency slots with producer recovered (1)	5,531 / 5,746 (96.3%)	559 / 881 (63.5%)
Episodes yielding an admissible window (2)	1,207 (85.3%)	37 (17.5%)
Families reaching the support floor (2)	32	19
Families that synthesize a program (4)	12	2
Held-out replay: pass / abstain / **wrong** (5)	24 / 12 / **0**	0 / 2 / **0**
Largest family support $n_{\max}$ (6)	26	8
Groups required by Eq. (7) (6)	92	92
Artifacts admitted	**0**	**0**

5.2 Cross-repository, time-forward extension

Can a guarded pre-model runtime survive beyond one repository when the task is exact and time-forward? Yes, but with claim boundaries that matter (table 2). In the five-repository cohort, discovery reaches 580/580 exact traces, four repositories complete 120/120 exact held-out pairs, and `pytorch/pytorch` retires at compile time rather than being forced through a looser threshold; requests fall 44.4% and cost 48.6%, though a fixed two-read template is more efficient still on this simplified task.

The balanced three-repository rerun strengthens the generalization story: 360/360 exact discovery traces and 180/180 exact held-out pairs, with requests down 66.7%, total tokens 78.4%, wall latency 60.7%, and estimated cost 72.7% (paired Wilcoxon $p < 10^{-27}$ on every resource endpoint). Yet the fixed template is essentially tied — same 180 provider requests, $0.0190 against the compiled artifact's $0.0188. The supported conclusion is not that learning dominates engineering on this task, but that the guarded compiler can automatically discover, validate, and transfer a useful artifact while retaining retirement as a valid output.

5.3 Refusal is a scientific result

NESTFUL and API-Bank show the opposite regime, and table 3 isolates which stage of Alg. 1 binds. On NESTFUL, provenance places the expected producer in the candidate set for 5,531 of 5,746 dependency slots (96.3%), 85.3% of episodes yield an admissible window, 12 of 32 recurrent families synthesize, and held-out replay yields 24 passes, 12 abstentions, and zero wrong executions — every earlier stage succeeds. The maximum family support is 26, while eq. (7) requires $n_\eta \geq 92$ zero-violation groups at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$, so calibration and only calibration retires every family. API-Bank fails earlier and harder: the effect catalog blocks 28 of 49 tools as writes or non-replayable before mining begins, leaving 19 families at maximum support 8; two synthesize, both abstain, and every family retires again.

This separates two properties that are routinely conflated: provenance reachability and replay correctness are properties of a *trace*, whereas admissibility is a property of the *evidence*. A system that shipped on the first two would have deployed twelve NESTFUL artifacts backed by 26 groups

of support. These results are the intended behaviour of eq. (4) when the sample cannot certify the constraint, not failures of implementation hygiene.

6 Related Work

Tracing JITs, guards, and deoptimization. GAC is structurally a tracing just-in-time compiler for agent executions, and the vocabulary nearly transfers: hot-trace detection becomes family mining, guard synthesis becomes the hard guard H, region compilation becomes bounded synthesis, and deoptimization becomes fallback to the baseline agent. Dynamo introduced transparent guarded trace regions with fallback (Bala et al., 2000); trace-based type specialization (Gal et al., 2009) and meta-tracing (Bolz et al., 2009) developed the guard-and-recompile discipline we borrow; and restoring a consistent state after abandoning a region is exactly dynamic deoptimization (Hölzle et al., 1992). Specializing against known inputs is otherwise partial evaluation (Jones et al., 1993). What is *not* inherited is what an agent guard must additionally certify: that arguments are grounded in observable state (eq. (1)), that declared effects permit speculation — effect systems supply the discipline for treating declarations as a trusted computing base (Lucassen & Gifford, 1988) — and that the residual error rate carries a finite-sample bound.

Trace-based agent optimization. The closest systems reuse or compile observed agent behaviour. AWO bundles recurring tool-call sequences into meta-tools, cutting LLM calls by up to 11.9% (Abuzakuk et al., 2026); Agent Workflow Memory induces reusable workflows for web tasks (Wang et al., 2025); Agentic Compilation emits a deterministic JSON workflow from one model call and replays it without further inference (Chundru, 2026); agentic plan caching adapts plan templates for 46.6% mean cost reduction (Zhang et al., 2025). The closest comparators are EvoC2F, which compiles a typed plan IR carrying dependency, effect, resource, idempotency, and retry annotations and admits macro-skills through tests and regression checks (Wei et al., 2026), and Agent JIT Compilation, which validates generated code plans under tool pre/post-state invariants (Winston et al., 2026). The distinction throughout is what *licenses* deployment: each admits on a metric, a profile, or a test suite, none on a finite-sample selective bound. We did not reimplement them, so we claim a different admission rule, not superiority.

Prompt, model, and topology optimization. DSPy and MIPRO optimize instructions and demonstrations in declarative LM programs (Khattab et al., 2024; Opsahl-Ong et al., 2024); GEPA evolves prompts reflectively from execution traces (Agrawal et al., 2026); RouteLLM selects a model from preference data (Ong et al., 2025); AgentSlimming prunes multi-agent graph nodes (Chen et al., 2026); FlowCompile searches model, budget, and topology for a *declared* graph (Li et al., 2026); LLMCompiler parallelizes the calls of the current request (Kim et al., 2024); and COVENANT compiles natural-language policy into checkable workflow graphs (Wang et al., 2026). GEPA in particular shows that trace-driven agent optimization is not itself novel, so we do not claim it. The difference is the object changed: these methods tune prompts, models, or the scheduling of calls the agent is *about* to make, whereas GAC deletes historical model boundaries when a trace-grounded deterministic replacement is supported by guarded evidence.

Program analysis and selective risk. Canonical trace families resemble process-mining variants (van der Aalst, 2016); bounded synthesis relates to programming by example (Gulwani, 2011), bounded sketching (Solar-Lezama et al., 2006), and likely-invariant mining (Ernst et al., 2001); argument grounding is execution provenance (Cheney et al., 2009). Admission builds on exact binomial inference (Clopper & Pearson, 1934) and distribution-free risk control (Angelopoulos et al., 2022; Bates et al., 2021), and the reject option goes back to Chow (1970). Our contribution is not a new bound but the observation that a compiler's natural calibration unit is the scenario *group*, and that the resulting sample requirement usually binds (section 5.3).

Tool-use evaluation and caching economics. Most tool-use suites test whether an agent can choose and execute actions (Patil et al., 2025; Lu et al., 2025; Barres et al., 2025; Qin et al., 2023; Liu et al., 2023; Mialon et al., 2023; Wei et al., 2025; Yoran et al., 2024; Jimenez et al., 2024); we use NESTFUL and API-Bank instead because they retain the nested producer references and recorded call results a post-trace provenance question requires. On economics, prompt-cache strategy has

been measured at 41–80% cost reduction, with naive full-context caching sometimes *increasing* latency (Lumer et al., 2026; Song, 2026); our observation that fewer tokens can cost more is consistent with that, arising here from turn removal rather than compression. Its deployed counterpart is context-compression middleware such as Headroom, which reports 73% token reduction on GitHub triage (Headroom Labs, 2026) by shrinking the payload of calls that still happen: it never removes a model boundary and so cannot reduce provider requests, where GAC does the opposite. The two compose; we did not run the composition, which is the comparator our token numbers most need. Finally, counterfactual trace auditing shows that unchanged aggregate pass rates can hide substantive behavioral differences (Zhou et al., 2026) — precisely the weakness of the registered-contract oracle we use.

7 Discussion and Limitations

The strongest supported claim is narrow but meaningful: repeated read-only workflow prefixes can be specialized into guarded deterministic programs that preserve exact held-out contracts on several real GitHub families while failing closed when evidence is inadequate — stronger than a token-savings report, because the same artifact retains principled retirements on NESTFUL, API-Bank, and one cross-repository cohort.

Gate maturity is the main scientific risk. Proposition 1 is per-candidate, not compiler-wide over the adaptive family search of Alg. 1, so the certificate is conditional on freezing the candidate relative to calibration. The gate is also step-like empirically: it behaves as an all-or-none support test at $n_\eta \geq 92$ rather than tracing a risk–coverage frontier, so we have shown that the rule *refuses* correctly far better than that *q discriminates*. Refitting q so the registered gate yields a genuine frontier is the highest-value follow-up.

External validity and practical competitiveness. The richer workflow-family evidence comes from one revision-pinned repository snapshot and one provider/model family, and the cross-repository extension broadens scope only on a narrower task where a fixed template is essentially tied; we establish no superiority on full cross-repository workflows, under drift, or across providers. Hand-written programs also remain competitive when the workflow is already known, and we do not measure manual authoring or maintenance cost — a complete comparison would price the engineering axis too.

Semantic scope. Exact task contracts do not certify semantic equivalence of the final answer once a model continuation follows; counterfactual trace auditing (Zhou et al., 2026) would detect the gap. Writes, handoffs, hosted tools, and undeclared effects sit outside proposition 1 entirely, and removing a model turn can remove a guardrail evaluation that ran at that turn — boundaries that are the conditions for an honest guarded deployment story, not accidents of the implementation.

8 Conclusion

GAC treats recurrent traces as candidate evidence, not permission to rewrite an agent. Across three live GitHub workflow families and a cross-repository extension, compile-or-retire yields substantial resource savings while preserving exact held-out contracts; on NESTFUL and API-Bank it refuses entirely. Judge workflow optimizers not only by the turns they remove, but by the evidence they require, what they refuse, and the baseline they fall back to.

AI Use Statement

In preparing this submission, the authors used generative AI tools to help restructure the manuscript, improve readability, suggest section organization, draft and revise parts of the paper text, and convert repository-grounded results into LaTeX tables and submission materials. We did not use generative AI tools to generate experimental results or to replace manual verification of claims. All reported numbers, citations, equations, and comparisons were checked against the repository artifacts, retained outputs, and source files, and the authors take responsibility for the final content of the paper.

Ethics Statement

This paper studies methods for removing model-mediated decisions from tool-using agents. The main risk is over-trusting a compiled prefix and thereby accelerating unsafe automation or silently removing safeguards; deleting a model boundary can also delete a guardrail evaluation that happened to run at that boundary. Our method mitigates this risk by compiling only read-only prefixes, treating unknown effects, approvals, handoffs, and writes as hard barriers, and defaulting to retirement or fallback when evidence is insufficient. All study data are public records (GitHub repository metadata) or public benchmarks; no personal or private data were collected. The reported artifact is not a license to bypass human review, legal or compliance controls, or safety guardrails in high-stakes domains; unsupported regimes are part of the result, not exceptions to hide.

Reproducibility Statement

The main paper reports exact cohort sizes, evaluation protocols, quantitative metrics, and the claim boundaries attached to each study. Algorithms 1 and 2 give the deployed compiler and admission rule; Algorithms 3 and 4 in the appendix give provenance construction and runtime dispatch. Appendix A proves Proposition 1, and Appendix C lists every hyperparameter used by the reported runs. The repository contains source manifests, raw result files, deterministic table and figure generation, and scripts for the live GitHub studies, the cross-repository extension, NESTFUL, and API-Bank. For anonymous submission, the code and retained artifacts should be packaged as an anonymous supplementary archive with hashes and manifests preserved.

References

Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026. URL https://arxiv.org/abs/2601.22037.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.19457. Oral presentation.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=TNcOrox3tQ.

Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. Dynamo: A transparent dynamic optimization system. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 1–12, 2000. doi: 10.1145/349299.349303.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL https://arxiv.org/abs/2506.07982.

Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33538–33547, 2025. doi: 10.18653/v1/2025.emnlp-main.1702. URL https://aclanthology.org/2025.emnlp-main.1702/.

Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6), 2021. doi: 10.1145/3478535. URL https://arxiv.org/abs/2101.02703.

Carl Friedrich Bolz, Antonio Cuni, Maciej Fijałkowski, and Armin Rigo. Tracing the meta-level: PyPy's tracing JIT compiler. In *Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS)*, pp. 18–25, 2009. doi: 10.1145/1565824.1565827.

Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. AgentSlimming: Towards efficient and cost-aware multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 30064–30086, 2026. doi: 10.18653/v1/2026.acl-long.1387. URL https://aclanthology.org/2026.acl-long.1387/.

James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi: 10.1561/1900000006.

C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.

Jagadeesh Chundru. Agentic compilation: Mitigating the LLM rerun crisis for minimized-inference-cost web automation, 2026. URL https://arxiv.org/abs/2604.09718.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993. doi: 10.1201/9780429246593.

Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.

Andreas Gal, Brendan Eich, Mike Shaver, David Anderson, David Mandelin, Mohammad R. Haghighat, Blake Kaplan, Graydon Hoare, Boris Zbarsky, Jason Orendorff, Jesse Ruderman, Edwin W. Smith, Rick Reitmaier, Michael Bebenita, Mason Chang, and Michael Franz. Trace-based just-in-time type specialization for dynamic languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 465–478, 2009. doi: 10.1145/1542476.1542528.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 317–330, 2011. doi: 10.1145/1926385.1926423.

Headroom Labs. Headroom: Compressing tool outputs, logs, and retrieved context before they reach the LLM. Software repository, https://github.com/headroomlabs-ai/headroom, 2026. Apache-2.0; self-reported 60–95% token reduction on JSON payloads and 73% on a GitHub triage workload. Accessed 7 August 2026.

Urs Hölzle, Craig Chambers, and David Ungar. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 32–43, 1992. doi: 10.1145/143095.143114.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993. URL https://www.itu.dk/people/sestoft/pebook/.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=sY5N0zY5Od.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 24370–24391, 2024. URL https://proceedings.mlr.press/v235/kim24y.html.

Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. FlowCompile: An optimizing compiler for structured LLM workflows. *arXiv preprint arXiv:2605.13647*, 2026. URL https://arxiv.org/abs/2605.13647.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023. URL https://arxiv.org/abs/2308.03688.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988. doi: 10.1145/73560.73564.

Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don't break the cache: An evaluation of prompt caching for long-horizon agentic tasks, 2026. URL https://arxiv.org/abs/2601.06007.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12:153–157, 1947. doi: 10.1007/BF02295996.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023. URL https://arxiv.org/abs/2311.12983.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024. URL https://arxiv.org/abs/2406.11695.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL https://arxiv.org/abs/2307.16789.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 404–415, 2006. doi: 10.1145/1168857.1168907.

Yan Song. Cache-aware prompt compression: A two-tier cost model for LLM API caching, 2026. URL https://arxiv.org/abs/2607.15516.

Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2 edition, 2016. doi: 10.1007/978-3-662-49851-4.

Jincheng Wang, Min Zheng, and Tao Wei. COVENANT: Natural-language workflow compilation for aligned agent execution. *arXiv preprint arXiv:2607.25400*, 2026. URL https://arxiv.org/abs/2607.25400.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 63897–63911, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025. URL https://arxiv.org/abs/2504.12516.

Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. EvoC2F: Compiling tool orchestration for efficient and evolvable LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://openreview.net/forum?id=ZSGB91kMOG.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. Agent JIT compilation for latency-optimizing web agent planning and scheduling. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2605.21470.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 8938–8968, 2024. doi: 10.18653/v1/2024.emnlp-main.505. URL https://aclanthology.org/2024.emnlp-main.505/.

Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Cost-efficient serving of LLM agents via test-time plan caching. *arXiv preprint arXiv:2506.14852*, 2025. URL https://arxiv.org/abs/2506.14852.

Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu. Counterfactual trace auditing of LLM agent skills, 2026. URL https://arxiv.org/abs/2605.11946.

Algorithm 3 BUILDPATG — typed argument provenance. A slot with no witness is a genuine model decision; a slot with too many witnesses is ambiguous. Both block the region rather than defaulting to the cheapest explanation.

Input: episodes $\mathcal{T}$; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ
Output: provenance graph G with one edge per grounded argument slot

$G \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \textsc{Flatten}(z),\ \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all tool calls $c_j \in e_{1:T}$ in order **do**
    for all argument slots $u \in \text{args}(c_j)$ **do**
      $\mathcal{H} \leftarrow \{\, \sigma \in \Sigma : \exists\, g \in \mathcal{L},\ g(\sigma) = u,\ \text{depth}(g) \le 2 \,\}$
      if u is declared *literal-only* for c_j **then**
        $\mathcal{H} \leftarrow \{\sigma \in \mathcal{H} : g = \text{id or } g \text{ constant}\}$ ▷ never reconstruct a page index or a limit
      end if
      if $\mathcal{H} = \emptyset$ **then**
        mark c_j MODEL-ORIGINATED; **block** region
      else if $|\mathcal{H}| > \kappa$ **then**
        mark c_j AMBIGUOUS; **block** region
      else
        $G \leftarrow G \cup \{\sigma \xrightarrow{g} (c_j, u)\}$ for the minimum-description-length σ
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \textsc{Flatten}(o_j)\}$ ▷ a result witnesses only *later* slots
  end for
  for all pairs (e_s, e_t) sharing a resource where either writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge: no reordering is ever proposed across a conflict
  end for
end for
return G

A Proof of Proposition 1

Fix one threshold $\eta \in \Lambda$ and write $\gamma = \delta/|\Lambda|$. Conditional on $n_\eta = m > 0$ admitted calibration groups, the number of violating groups k_η is binomial with success probability equal to the selective risk r_η at that threshold, because the candidate program, guard, verifier, score, and grid were all frozen before any calibration group was observed and admitted groups are i.i.d. Inverting the one-sided exact binomial test (Clopper & Pearson, 1934) gives the upper bound $U_\eta = \text{Beta}^{-1}(1-\gamma;\ k_\eta + 1,\ m - k_\eta)$ of eq. (7), which satisfies $\Pr\{r_\eta > U_\eta \mid n_\eta = m\} \le \gamma$. When $m = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives $\Pr\{\exists \eta \in \Lambda : r_\eta > U_\eta\} \le |\Lambda|\gamma = \delta$, i.e. simultaneous coverage with probability at least $1 - \delta$. On that event, any threshold whose upper bound is at most α also satisfies the target selective risk of eq. (4), including the coverage-maximizing threshold returned by Alg. 2. ■

Two scope conditions are worth restating. First, the statement is *per fixed candidate*: Alg. 1 searches over families and returns the first admissible one, and that outer search is not multiplicity-corrected. Second, i.i.d. admitted groups is an assumption about the calibration cohort, not a property the compiler verifies; the grouped split in Alg. 1 makes it plausible by construction but does not establish it under temporal drift.

B Additional Algorithms

Algorithm 3 constructs the typed provenance graph that decides eq. (1). Its two block conditions are asymmetric on purpose: a slot with no witness is treated as a genuine model decision and a slot with more than κ witnesses is treated as ambiguous, so the analysis never resolves an argument by picking the cheapest available explanation.

Algorithm 4 is the runtime counterpart of eq. (3). Cheap deterministic checks reject first and the calibrated gate runs only on what survives them, so the common case costs a registry lookup and

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; the rest return the unmodified agent, and only a dirty post-commit failure raises an incident.

Input: entry state z; runtime context M'; registry $\mathcal{R}$; mode $m \in \{\mathsf{off}, \mathsf{shadow}, \mathsf{live}\}$
Output: observations for the region, or FALLBACK to the baseline agent B

if $m = \mathsf{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}.\text{RESOLVE}(M'.\text{compatibility}, M'.\text{partition})$
if $A = \bot \lor A.\text{lifecycle} \neq \text{ACTIVE} \lor \neg\text{VERIFYSIGNATURE}(A)$ **then**
  return FALLBACK
end if
if $H_A(z, M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if $\text{tools}(A.P) \cap \text{ALREADYOBSERVED} \neq \emptyset$ **then**
  return FALLBACK ▷ the region already ran; its live-ins are not the fitted ones
end if
if $m = \mathsf{live} \land \exists f \in \text{tools}(A.P) : C(f).\mathit{quota\_attested} \land \text{SNAPSHOT} = \bot$ **then**
  return FALLBACK ▷ no staging owner, so a discarded attempt is not attestable
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \mathsf{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behaviour
end if
$S \leftarrow \text{STAGE.BEGIN}()$; $r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$ ▷ the facade re-checks effects at execution time
if r raised PRECOMMITERROR **then**
  assert $S.\text{REVERSIBLE}()$; **return** FALLBACK
else if r raised POSTCOMMITERROR **then**
  return INCIDENT ▷ an external commitment happened; do not feign rollback
end if
if $V_A(r) \neq 1$ **then**
  discard r; **return** FALLBACK ▷ live-outs, effect multiset, and call count are checked before use
end if
$S.\text{COMMIT}(r.\text{effects})$; **return** $r.\text{observations}$

a guard evaluation. Exactly one exit path compacts; five return the unmodified baseline agent, and one — a post-commit failure — raises an incident rather than feigning a rollback. "Baseline" is byte-exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas a model-boundary adapter cannot un-emit a response the host has already committed to conversation history.

C CONFIGURATION

All reported admission runs use risk budget $\alpha = 0.05$, confidence budget $\delta = 0.1$, and a fixed grid of $|\Lambda| = 11$ thresholds, which fixes the zero-violation sample requirement at $n_\eta \geq \log(\delta/|\Lambda|)/\log(1-\alpha) = 91.6$, i.e. 92 admitted groups; at exactly $n_\eta = 92$ and $k_\eta = 0$ the bound is $U_\eta = 1 - (0.1/11)^{1/92} = 0.0498 \leq \alpha$. Tightening the confidence budget to $\delta = 0.05$ would raise the requirement to 106 groups, which no study in this paper reaches, so $\delta = 0.1$ is the operative setting throughout and is reported as such rather than rounded to match α. Synthesis uses a 23-operator closed library at depth ≤ 2; region mining uses $w_{\max}$ tool calls per window and requires support across independent scenario groups. The implementation also supports a minimum-distinct-days requirement, which the single-snapshot live study does not exercise (`min_days = 1`). Two terms of eq. (5) are coarser in the implementation than the notation suggests: ρ_{eff} is approximated by the fraction of tool names containing a namespace separator rather than read from the effect catalog, and $|F|$ is substituted by the argument-slot count of the first observed window because ranking precedes

synthesis. Both appear only in a ranking heuristic — no admission decision depends on either, and the effect catalog is consulted where it is load-bearing, in qualification and in the runtime facade.

D Paired Statistics for the Primary Families

Per-record paired differences (compiled minus baseline) on the two families that admit a deep prefix, with 10,000-sample bootstrap intervals and two-sided Wilcoxon signed-rank tests over $n = 30$ paired held-out records:

Family	Endpoint	Paired difference (95% CI)	p
PR-outcome	Total tokens	-2196.7 [-2211.1, -2182.5]	1.7×10^{-6}
	Wall latency	-3889 ms [-4499, -3346]	1.9×10^{-9}
	Estimated cost	$-\$5.11 \times 10^{-4}$	1.7×10^{-6}
Backlog	Total tokens	-2315.0 [-2366.0, -2224.1]	1.7×10^{-6}
	Wall latency	-4428 ms [-5400, -3558]	1.9×10^{-9}
	Estimated cost	$-\$5.44 \times 10^{-4}$	1.7×10^{-6}

Provider-request differences are deterministic by construction (every pair removes the same number of turns), so no rank test is reported for them: the statistic would measure the degeneracy rather than the effect.

E Additional Experimental Detail

Primary workflow families. The three-family result is the main paper's strongest live evidence because it combines distinct tool vocabularies, exact task graders, balanced held-out cohorts, and provider-backed execution. The issue-type family intentionally retains a shallower two-read prefix; the newer pull-request and backlog families admit deeper pre-model compaction. This difference explains the lower resource savings on the first family and is part of the evidence boundary, not noise to average away.

Cross-repository extension. The cross-repository study is narrower than the primary workflow families. Its task uses two read-only tools and a simpler exact outcome contract, which is precisely why a fixed template can remain tied or even more efficient than the learned artifact. We keep the result in the main text because it addresses a reviewer-critical scope question—whether guarded specialization survives beyond one repository—while being explicit about the reduced task complexity.

The position invariant, empirically. An early pilot dispatched a compiled *suffix* region at the entry boundary, violating eq. (2). Because the runtime resolves artifacts before the agent has made the calls the region assumed had already happened, the pilot reordered and duplicated tool calls. The failure is a semantic mismatch between compilation-time position and resolution-time position, not a tuning error, which is why eq. (2) is a hard constraint in the deployed compiler rather than a scoring penalty.

Omitted from the main claim. The repository retains additional studies that are useful scientifically but less suitable for the main narrative:

- a natural-order issue study that exposed a factual continuation miss for a more aggressive three-read artifact,
- exploratory guarded composite synthesis and a follow-up comparison to a fair pre-model manual program,
- a bounded GEPA prompt-optimization study in which GEPA retained its seed, so the combined arm is a replication rather than evidence of interaction, and
- a portfolio pilot and controlled demonstration suite for runtime boundary coverage, including workloads where removing turns *increases* cost through prefix-cache fragmentation.

These studies remain important to the broader artifact, but they either use a weaker risk level, address a narrower engineering question, or are explicitly exploratory.

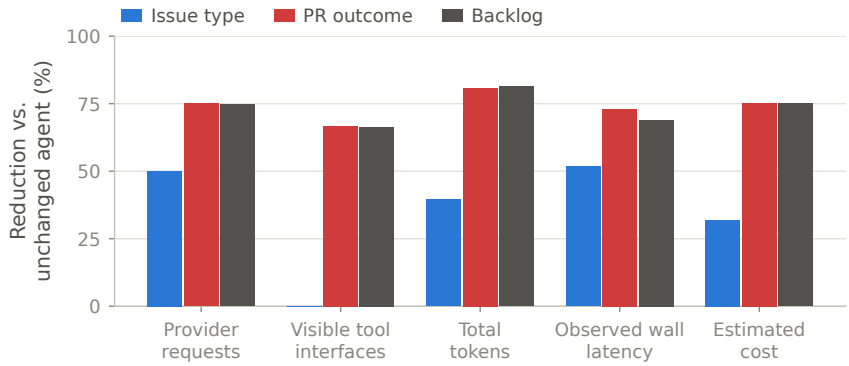

Figure 3: Per-family resource reductions behind table 1, plotted because the spread is the result: admissible prefix depth — not the optimizer — sets the savings, and the two deeper families sit at the feasibility ceiling of eq. (6).

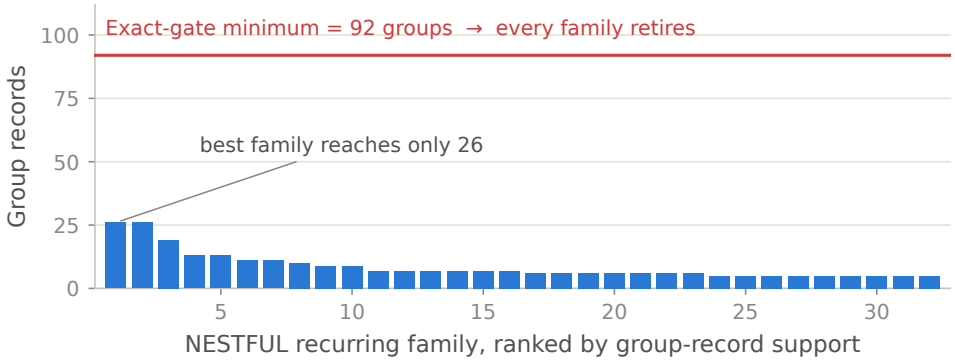

Figure 4: NESTFUL family supports against the 92-group requirement implied by eq. (7) at $\alpha = 0.05$, $\delta = 0.1$, $|\Lambda| = 11$. The largest family reaches 26 groups, so every family retires at calibration even though provenance, synthesis, and replay all succeeded.

F Extended Limitations

The current empirical gate does not yet demonstrate a useful risk–coverage frontier at the registered 5% level. The strongest current runs are mostly all-or-none support tests (fig. 4), and the compiler-wide candidate-search problem is not yet multiplicity-corrected. Likewise, exact task contracts do not certify full semantic equivalence of the final answer, especially once a downstream model continuation is involved. Two further operational caveats belong in the record: the headline live artifact was compiled without a sandbox available to the perturbation suite, so it records `perturbations_claimed: false` rather than a completed challenge, and registry signature verification was configured off in the reported run. These are not hidden defects in the repository; they are the reasons the paper avoids stronger deployment claims.

G Anonymous Artifact Release Plan

Before anonymous submission, package the code and retained artifacts as an anonymous supplementary archive preserving hashes, manifests, scripts, and raw results for the studies cited in the main paper. Direct links to the identified public repository should remain withheld until the review process permits deanonymization.